Unit 5. Complexity and regularization

Machine Learning 1 (ML1)

Master in Artificial Intelligence (MIA)

November 25, 2025

5.1 Bias and Variance, Underfitting vs Overfitting, and model complexity

The Bias-Variance tradeoff

If general, it can be shown that the **errors** made by learning algorithms can be decomposed in **bias** and **variance** terms.

For example, let's say that, in a regression problem, we use **Mean Squared Error** (MSE) to measure the error in the **prediction** $\hat{f}(x_0)$ of a test **instance** x_0 with respect to its **real value** y, where:

- $y = f(x) + \epsilon$
- f is the function we are trying to estimate.
- ϵ is random noise.

The **expected** error can the be expressed as:

$$E(y_0 - \hat{f}(x_0))^2 = Var(\hat{f}(x_0)) + Bias(\hat{f}(x_0))^2 + Var(\epsilon)$$

- $E(y_0 - \hat{f}(x_0))^2$ is the **expected test MSE**: the average test MSE that we would obtain if we **repeatedly generate models** $\hat{f}$ using a large number of training sets, and test each one with x_0.
- $Var(\hat{f}(x_0))$ is the **variance** of **predictions** made by the models.
- $Bias(\hat{f}(x_0))$ is the **difference** between the **average prediction** of the models and the **real value** which we are trying to predict.
- $Var(\epsilon)$, also called **irreductible error**, is the **variance** of the **noise** in the observed data.
  - We have no control over it, so we will ignore it in this discussion.

Bias is considered a **systematic error** in the learning algoritm due to **incorrect assumptions** in the ML process.

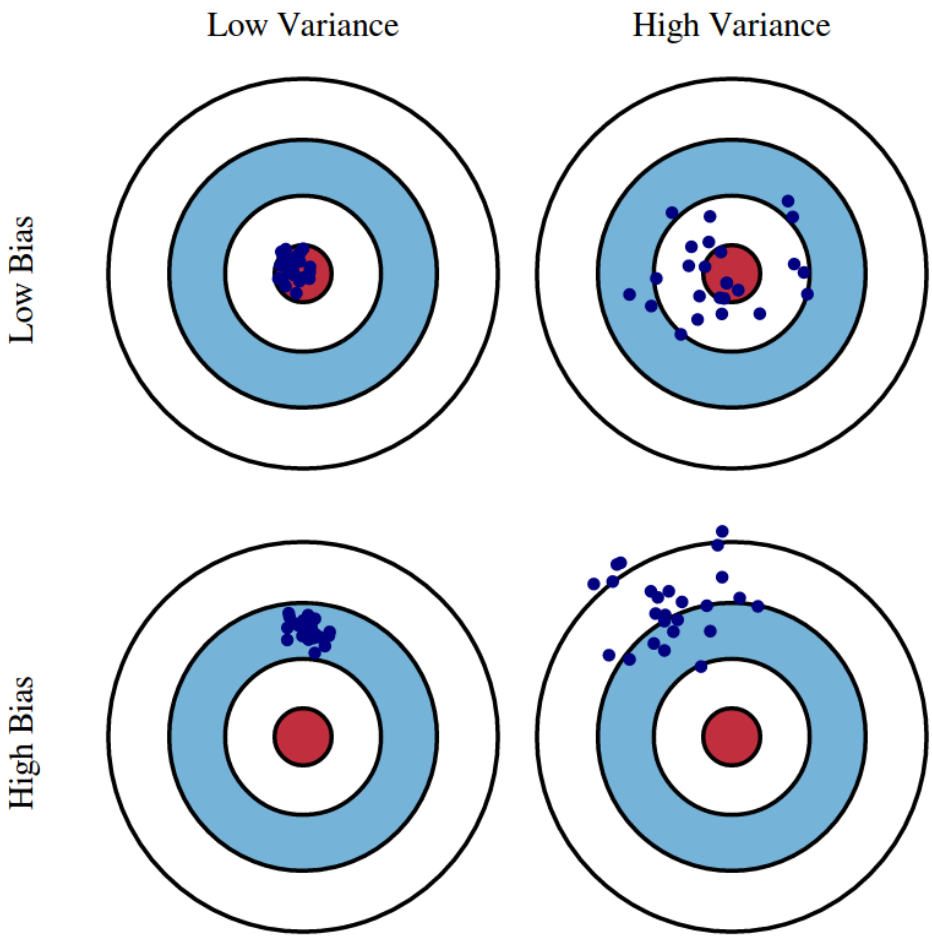
- Relationships in the data are not adequately captured leading to **oversimplification** of the resulting models.

Variance is an **error** from sensitivity to **small differences** in the training set.

- Small differences in the training data produce very different models.

A **graphical interpretation** of bias and variance:

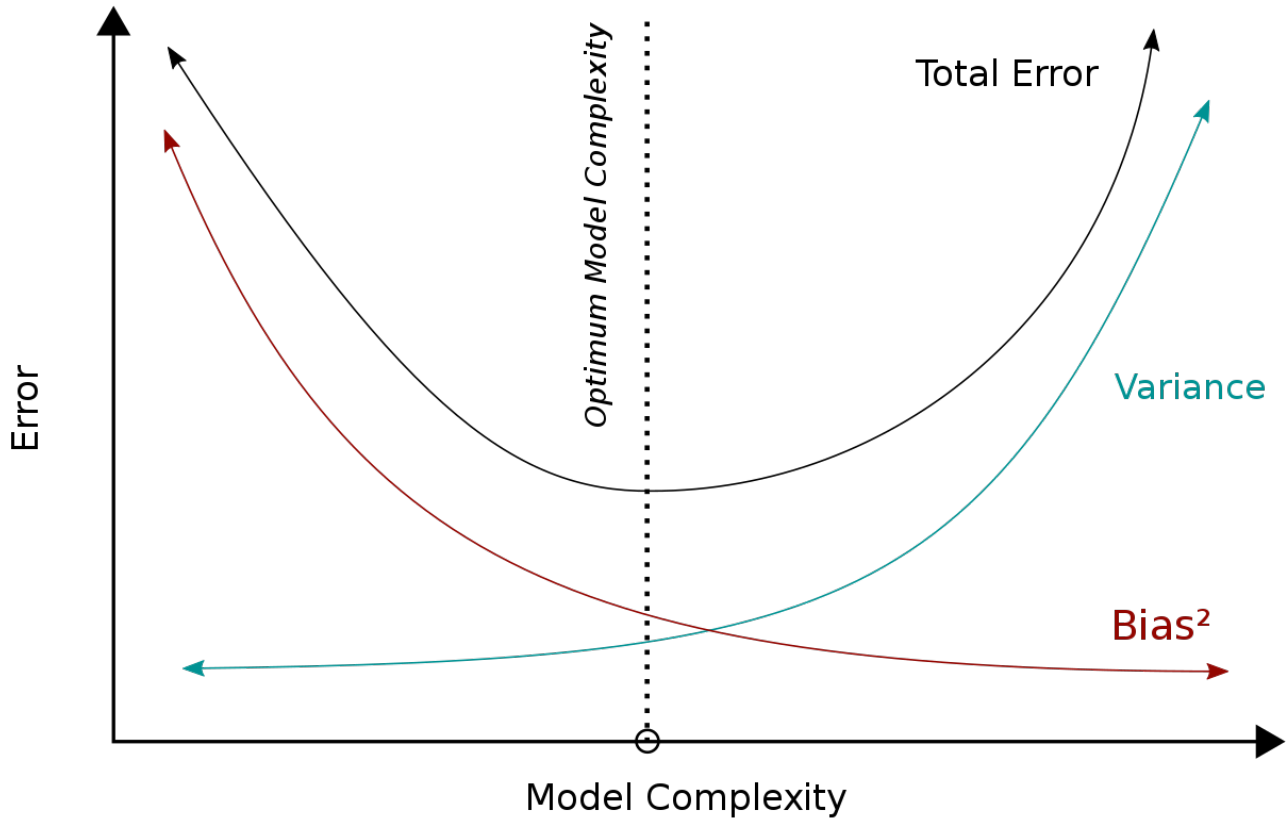
- Bullet holes are predictions of x_0 by different models trained by the same learning algorithm on different training sets.
- Bullseye is the real value.



Source: "Understanding the Bias-Variance Tradeoff", by Scott Fortmann-Roe

- **Low bias** and **low variance** $\Rightarrow$ predictions of x_0 from all models are tightly clustered around the real value.
- **Low bias** and **high variance** $\Rightarrow$ predictions are scattered around the target.
- **High bias** and **low variance** $\Rightarrow$ while predictions of x_0 are tighly clustered, they are not near the real value.
- **High bias** and **high variance** $\Rightarrow$ predictions are scattered all over the place.

The **bias-variance tradeoff** refers to the fact that both kinds of error depend on the complexity of the model.



Source: "Bias-variance tradeoff" (wikipedia)

Adding **complexity** (more hidden layers in an ANN, a higher order in a polynomial curve fitting, . . .) leads to **lower bias**.

- The resulting models are **more capable** of capturing the undelying relationships in the data.

But it also leads to **more variance**.

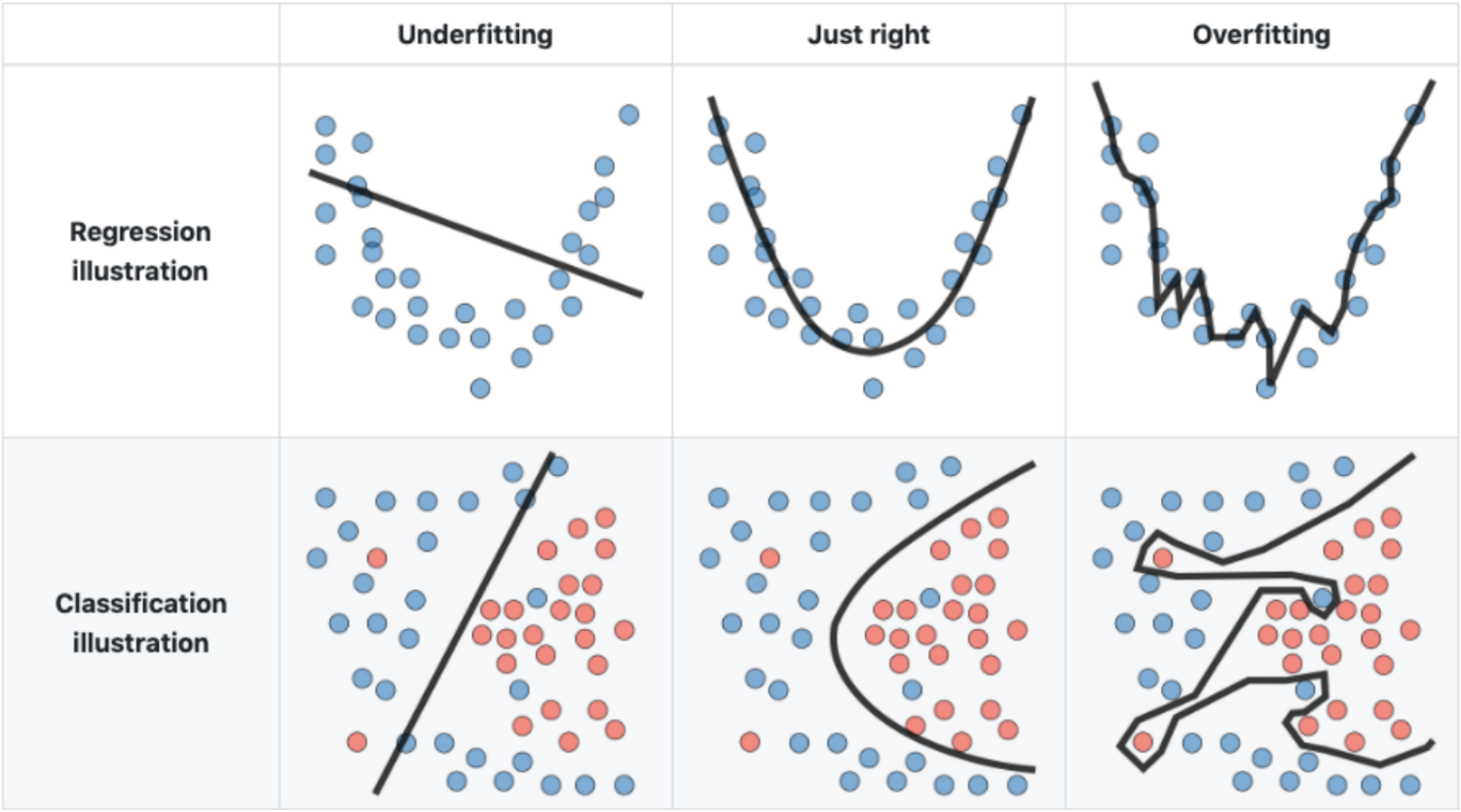
- Differences between training datasets are also captured by the models, even if they are due to **noise** in the data.

The **goal** is to hit the sweet spot in which bias and variance are both minimized, and **test error is minimal**.

Underfitting and overfitting

Two concepts tightly related to the previous discussion are **overfitting** and **underfitting**.

- **Underfitting** happens when the learning algorithm misses the relevant relations between features and target outputs ⇒ it does not learn well.
  - The resulting model presents **high error** in its predictions for both **training** and **test** data.
  - **High bias**, **low variance** (if several models are trained on slightly different data).
- **Overfitting** happens when the algorithm models very closely every detail in the training data, including random noise ⇒ it learns too well.
  - The resulting model is highly **accurate** on the **training data** but shows **poor** performance on **test data**.
  - It **memorizes** the training data ⇒ it does not generalize well.
  - **Low bias**, **high variance**.



Source: S. Omri, C. Sinz. Machine Learning Techniques for Software Quality Assurance: A Survey

- E.g., models with training error around 30% and test error around 35% exhibit **high bias**, but not necessarily a lot of **variance** $\Rightarrow$ **underfitting**.
- E.g., models with a 2% training error and 20% generalization error represent **low bias** and **high variance** $\Rightarrow$ they **overfit** the data.
- E.g., models showing training and test errors around $4-5\%$ show **low bias** and **variance** $\Rightarrow$ learning algorithm **generalizes well**.

Model complexity

Complexity has an efect in both **bias** and **variance**, and **overfitting** and **underfitting**, reflected in both **training** and **generalization error**.

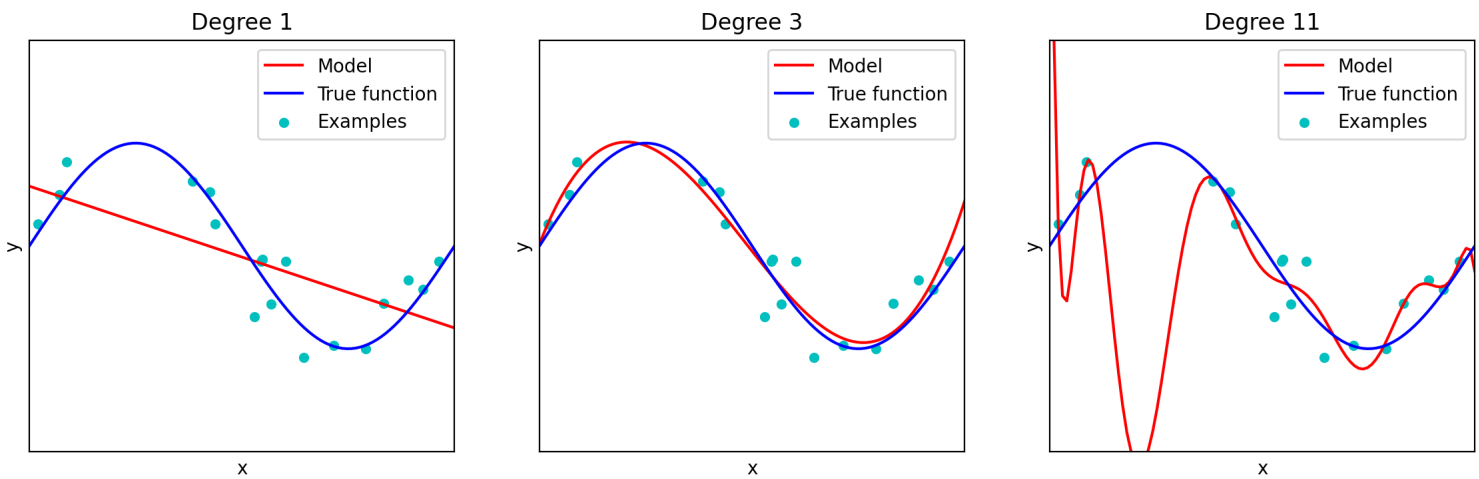
We will illustrate that effect with a simple example:

- We try to approximate $f(x) = \sin(2\pi x)$ with **polynomials** of different degrees (1, 3 and 11) via **linear regression**.

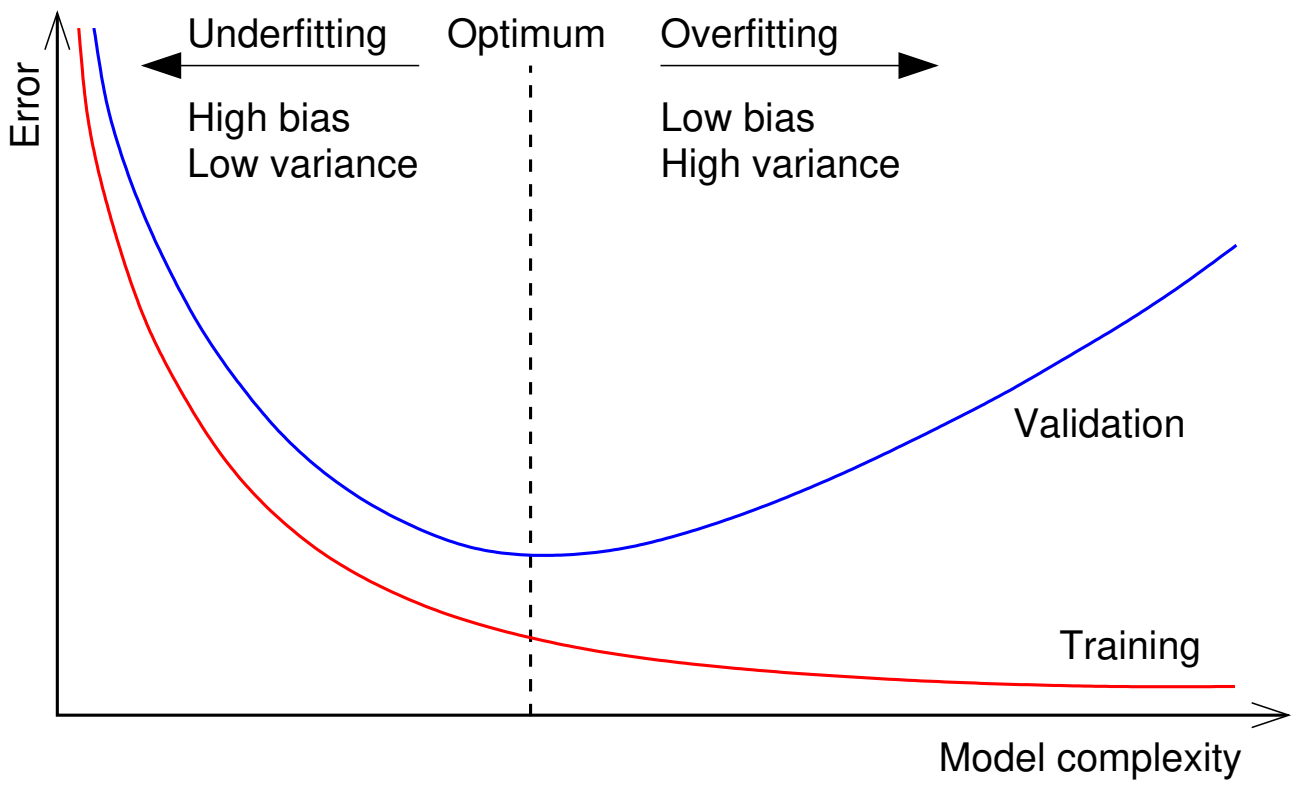
  The **degree d** of the polynomial controls model **complexity**.

  $$\hat{f}(x) = \theta_d x^d + \theta_{d-1} x^{d-1} + \ldots \theta_1 x_1 + \theta_0$$

  $d+1$ parameters (coefficients, weights), $\theta_i,\ 0 \leq i \leq d$ to estimate in the training process.
  - **Small values** of d $\Rightarrow$ less parameters, **simpler** shapes, **looser fit**.
    $d = 0 \Rightarrow$ horizontal line (constant function).
    $d = 1 \Rightarrow$ a straight line (linear function).
  - **High values** of d $\Rightarrow$ more parameters, more **complex** shapes, **tighter fit** to the training data (at least in theory).
- Training data: $n = 18$, sampled from $y = \sin(2\pi x) + \epsilon$ in $[0, 1]$

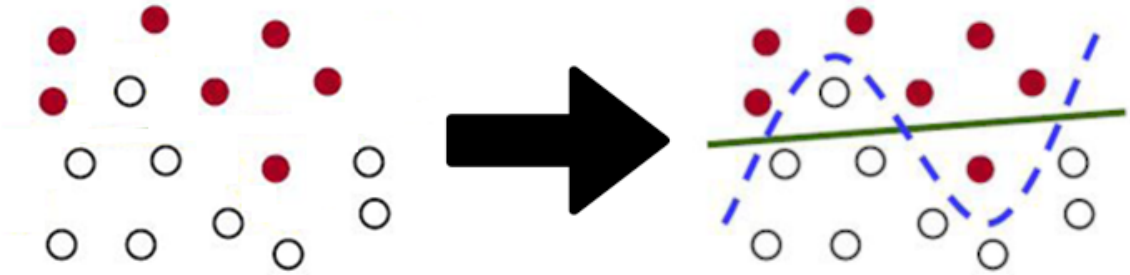


- Model **complexity** (degree d of the polynomial) has a clear effect on **performance**:
  - **Leftmost graph**: we try to fit the data using a **linear function**. Model **underfits** the training data $\Rightarrow$ too simple.
    - **High training** and **generalization errors**.
    - **High bias** and **low variance** $\Rightarrow$ small differences in training data will lead to very **similar** models.
  - **Middle figure**: **cubic polynomial**. **Right fit** for the data at hand $\Rightarrow$ complex enough, but not too much.
    - **Low training** and **generalization errors**.
    - **Low bias** and **variance**.
  - **Rightmost graph**: **degree 11 polynomial**. Severe **overfitting** $\Rightarrow$ model **memorizes** the data $\Rightarrow$ too complex.
    - **Low training error** and **high generalization error**.
    - **Low bias** and **high variance** $\Rightarrow$ small differences in training data will lead to very **different** models.



As we have just seen, avoiding overfitting or underfitting implies a choice of model related to the **complexity** of the learning algorithm.

- Usually our choice will be the **simplest algorithm** capable of generalize well from data.
- But, which one is that?



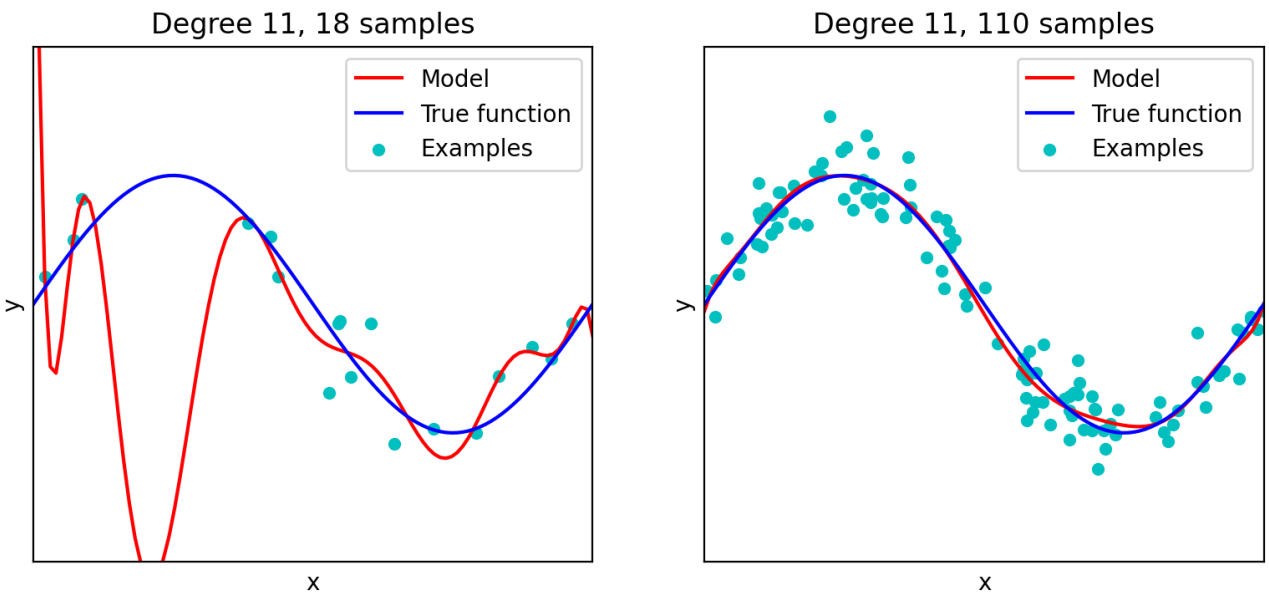
  - Simple model (green line) $\Rightarrow$ does not explain the data well.
  - Complex model (dashed blue line) $\Rightarrow$ may be overfitting the data.

To determine the **right type of model** a balance must be found between:

- **Model complexity**.
- **Size** of the available **dataset**.
- **Generalization error** (measured on a validation/test dataset).

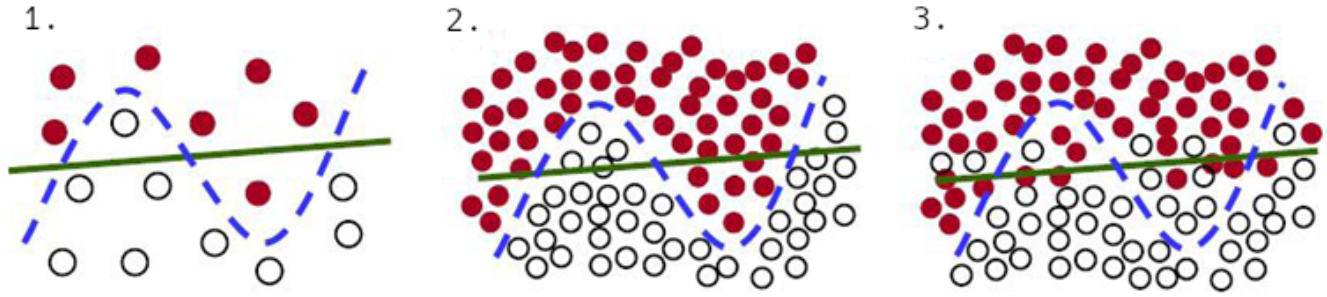
E.g., going back to the curve fitting of $f(x) = \sin(2\pi x)$.

- With $n = 18$ instances a $d = 11$ polynomial overfitted the data.
- Let's see what happens if we increase n to 110.



- The new model is a much better fit, thanks to the additional data.

Typically we will encounter three **scenarios**:



1. **Only a few instances**.
   - The **complex** model (dashed blue) is more **accurate** than the simple one (green), but **not by much**.
   - We may think that the **simpler** model generalizes **better**, because the misclassified instances may be noisy data.
   - **Problem**: we cannot really say if it is so, because we have only a handful of instances.
     Too few to know the influence of noise in misclassified examples.
2. **Many instances, no noise**.
   - The **complex** model (blue dash line) is **more accurate** than the simple one (green line) $\Rightarrow$ the complex model is **chosen**.
   - But it is **unlikely** scenario $\Rightarrow$ data is noisy in the real world.
3. **Many noisy instances**.
   - Both models seem to have **similar accuracy**.
   - We choose the **simpler** one (green line).

In the next sections, we will see:

- The theoretical justification of the relationship between model complexity, dataset size and test error.
- Regularization techniques to manage model complexity.

5.2 The Vapnik-Chervonenkis dimension

The **VC dimension** is a measure of the **complexity** (capacity, expressive power, richness, flexibility) of a family $f(x;\theta)$ of functions that can be learned for **binary classification**:

- f represents the **family of functions**.
- θ represents the **parameters** to be learned as part of the training process.

It is defined as the **cardinality** of the largest set of points that can be **explained** by the functions in $f(x;\theta)$

- Complexity is measured by the **number of examples** that a type of **classifier** is capable of correctly **separate** or "shatter".

The bigger the **complexity** of $f(x;\theta)$:

- the bigger the capacity to **fit** the model to an **increasing number** of training examples,
- at the expense of the resulting models being able to **generalize effectively** from training data.

Shattering of a set of examples:

- We have n examples $x_1, x_2, \ldots, x_n$ in a **binary** classification problem.
- Each example has d **real-valued features** $\Rightarrow$ it is a point in $\mathbb{R}^d$ (d-dimensional space).
- There exist 2^n possible ways to assign n examples to 2 classes, or **dicotomies**. E.g., 8 possible dicotomies for $n = 3$:

x_1	x_2	x_3	x_1	x_2	x_3	x_1	x_2	x_3	x_1	x_2	x_3	...	x_1	x_2	x_3
0	0	0	0	0	1	0	1	0	0	1	1	...	1	1	1

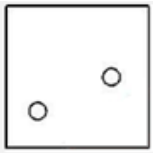
- A family $f(x;\theta)$ **shatters** the points (examples) if for **every possible dicotomy** exists a set of values for the parameters θ capable of separating the examples in it.
  - $f(x;\theta)$ can provide 2^n models, one for each possible dicotomy, with different parameters θ.

The **VC dimension** of $f(x;\theta)$ is the **maximum** number of points that can be arranged so that $f(x;\theta)$ **shatters** them.

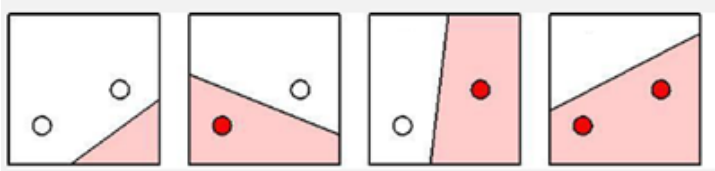
- More formally, the **maximum cardinal D** such that some data point set of cardinality D can be **shattered** by $f(x;\theta)$.
- If the VC dimension of $f(x;\theta)$ is D, there exists at least one set of D examples in $\mathbb{R}^d$ that can be shattered by $f(x;\theta)$.

For example:

- $n = 2$ and $d = 2$ (two points in a plane)

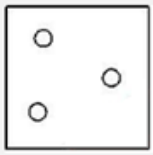


- $f(x;\theta)$ is the family of hyperplanes $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$ in $\mathbb{R}^2$ (linear functions), and $\theta = [\theta_0, \theta_1, \theta_2]$
- For each dicotomy we can find a different $\theta = [\theta_0, \theta_1, \theta_2]$ separating the examples.

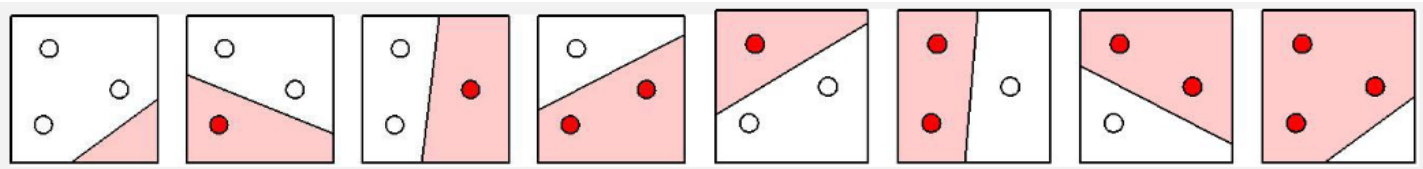


So the VC dimension of $f(x;\theta)$ is at least 2.

- $n = 3$ and $d = 2$ (three points in a plane)



- Again we can find a different $\theta = [\theta_0, \theta_1, \theta_2]$ separating the points in each dicotomy.



So the VC dimension of $f(x;\theta)$ is at least 3.

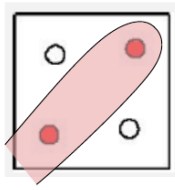
- That does not mean that every set of 3 points en $\mathbb{R}^2$ can be shattered by hyperplanes in $\mathbb{R}^2$.

  E.g., if the three points are aligned, there is a dicotomy for which they cannot be separated by a straight line.

- On the other hand, for $n = 4$ and $d = 2$ a set of points that can be shattered by $f(x;\theta)$ does not exists.

  There is always at least a dicotomy in which separation is not possible.

- So the VC dimension of two-dimensional hyperplanes is 3.
  - That is a low VC value $\Rightarrow$ low complexity.
- Another family of functions could have a dimension 4 or higher.
  - Is would separate a bigger number of examples in $\mathbb{R}^2$ regardless of how they are distributed.
  - It would be a more complex family of functions, maybe with lower generalization capabilities.
    E.g., polynomials of a degree $n > 1$.



The VC dimension allows us to calculate a **probabilistic upper bound** on the expected test error (**expected risk**) from the error on the training set (**empirical risk**) for classification models.

$$P\left(E_{test} \leq E_{train} + \sqrt{\frac{1}{n}\left[D\left(\log\left(\frac{2n}{D}\right)+1\right) - log\left(\frac{\eta}{4}\right)\right]}\right) = 1 - \eta$$

where:

- E_{test} is the error on the test set (**expected risk**).
- E_{train} is the error on the training set (**empirical risk**).
- D is the **VC dimension** of the family of functions $f(x;\theta)$.
- n is the number of **training examples**, and $\mathbf{n > D}$ **holds**.
- The term in red is called **VC confidence**, or **confidence interval**.
- The **sum** of E_{train} and the VC confidence is the **upper bound** of E_{test} with a **probability** of $1 - \eta$.

When the ratio $\boldsymbol{n/D}$ **grows**, the **VC confidence decreases** and the training error is a better approximation of the test error.

- It signals **less overfitting** $\Rightarrow$ **better generalization** capabilities.
- **Two possibilities**:
  1. The number of **training examples** $\boldsymbol{n}$ **grows**.
     - The bigger the number of training examples $\Rightarrow$ the bigger the generalization capabilities of the model (usually).
  2. The **VC dimension** $\boldsymbol{D}$ **decreases**.
     - Model can separate less examples $\Rightarrow$ is less complex.
     - Lower complexity leads to better generalization.

When the ratio $\boldsymbol{n/D}$ **decreases**, the **VC confidence increases** and empirical and expected risks grow apart.

- **More overfitting** $\Rightarrow$ **worse generalization** capabilities.
- Again **two possibilities**:
  1. The number of **training examples** $\boldsymbol{n}$ **diminishes**.
     - Less training examples $\Rightarrow$ less generalization power.
  2. The **VC dimension** $\boldsymbol{D}$ **grows**.
     - More complex model $\Rightarrow$ less generalization power.

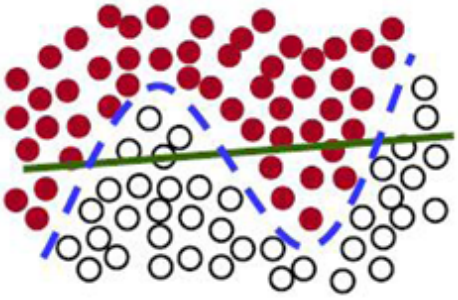
A **good model** is one in which the **expected risk** (test error) is **low**. Since it is the sum of empirical risk (training error) and VC confidence we want to minimize both:

- **Low training error** $\Rightarrow$ **little or no underfitting**.
- **Low VC confidence** $\Rightarrow$ **little or no overfitting**.

We can **prevent underfitting** by looking at the training error.

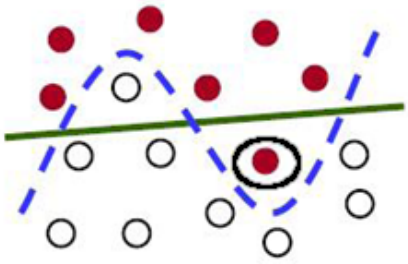
- The **training error** is **too high** $\Rightarrow$ we **need a more complex model** (polinomial with a higher degree, ANN with more hidden layers, ...).

  Model with linear decision boundary too simple $\Rightarrow$ underfits the data.
  We choose the more complex model (blue decision boundary).

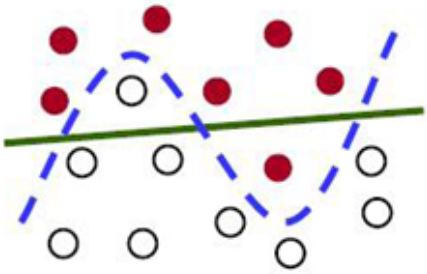


We can **prevent overfitting** by reducing VC confidence, i.e. growing n/D

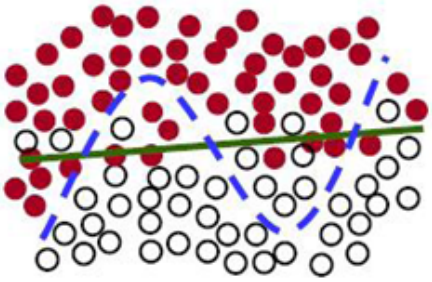
1. Having **more training examples** n.
   - The lower the value of n the higher the importance of each specific instance.
     - It could be the only one in a particular region of the feature space.
     - If that specific instance is put in the test set we are losing coverage of that particular region in training.



     - The more training instances the better, especially if the dataset is small $\Rightarrow$ cross validation, ensemble models, ...
   - Having more training instances dimishes the effects of noisy data.



Small dataset $\Rightarrow$ noise matters

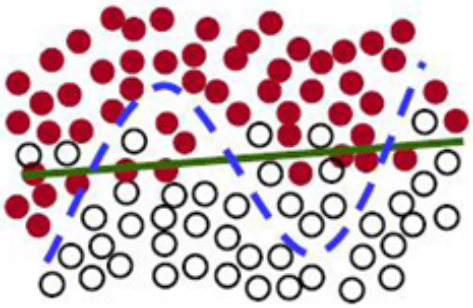


Bigger dataset $\Rightarrow$ effect of noise more muted

   - In many cases it will be impossible to get more examples.
     Fortunately there is a simpler alternative.

2. Choosing a **model with lower complexity**. Two possibilities:
   (a) Choose a **learner with a lower** D (VC dimension).
   The lower the complexity, the less the model will "learn" the noise in the data.

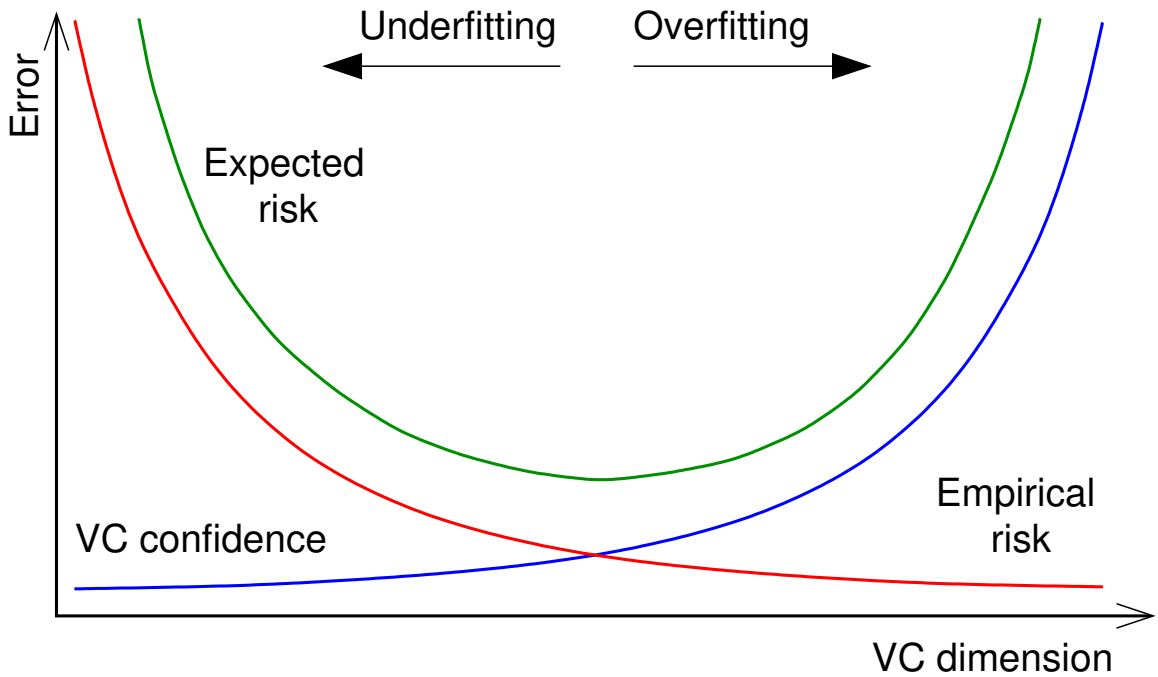
   Both models seem to have similar training error $\Rightarrow$ we will choose the simplest one.



   (b) **Regularization**: we keep a complex model, but we prevent it from memorizing the training data.

In summary:

- To **prevent underfitting** $\Rightarrow$ **increase complexity**.
- To **prevent overfitting** $\Rightarrow$ **decrease complexity**.
- Again, we are tring to find a "sweet spot" of complexity.

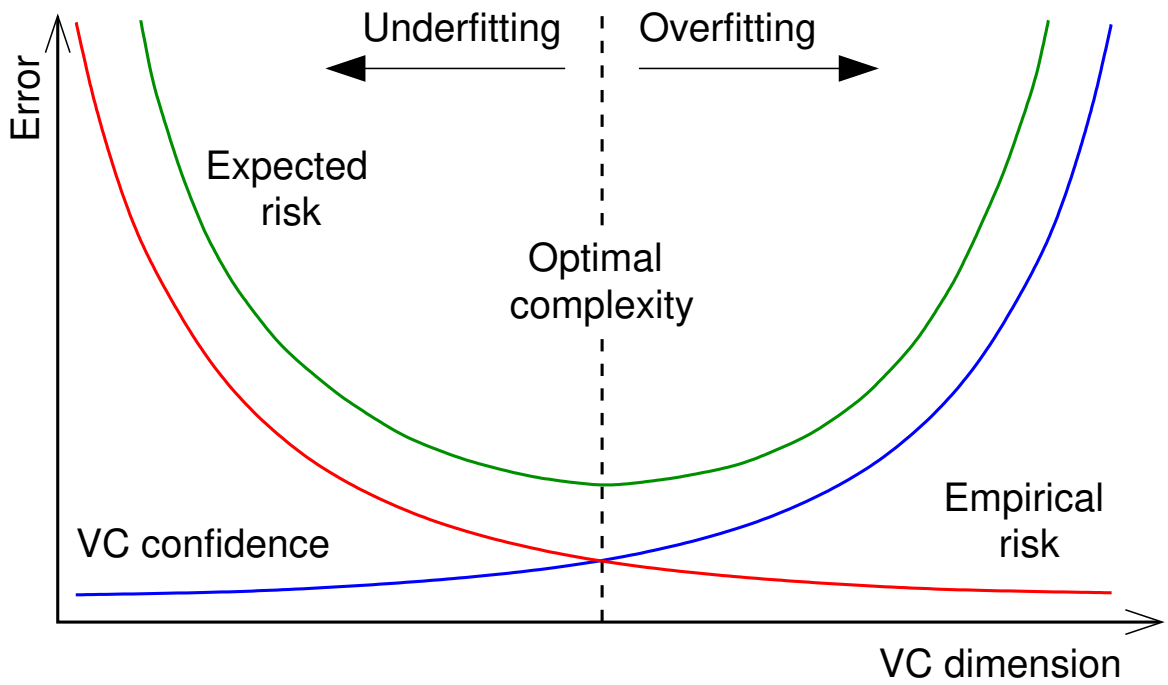


To search for an optimal model is to search for a **balance** between **empirical risk** and the **VC dimension** of the model.

- If **VC dimension too low**:
  - Model not complex enough ⇒ **underfitting**.
  - Low VC confidence ⇒ test error similar to training error.
  - But training error (empirical risk) will be high.
  - Expected risk (sum of empirical risk and VC confidence) will be high.
  - **High bias**, **low variance** (when several models are trained).
- If **VC dimension too high**:
  - Model too complex ⇒ **overfitting**.
  - Training error (empirical risk) low.
  - But high VC confidence ⇒ test error much higher than training error.
  - Expected risk (sum of empirical risk and VC confidence) will be high.
  - **Low bias**, **high variance**.

How to determine ideal complexity?

- In theory we want a model with a complexity in the lower bound of the expected risk.
  - For given n and D.



- In practice we cannot calculate the VC confidence in most situations.
- When it is possible, there are specific equations for the VC dimension of decision trees, ANNs, SVMs, boosting ensembles, . . .

5.3 Regularization

Regularization is a the general name of a series of techniques to **control complexity** in the learning process in order to **avoid overfitting**.

It is commonly done by adding a **term** to the learning objective dependent on the **weights** of the model in order to **minimize** them in the optimization process.

$$\arg\min_{\theta} \sum_{i=1}^{n} (L(y_i, f(x_i;\theta))) + \lambda\, R(\theta)$$

where

- $L(y_i, f(x_i;\theta))$ is the **loss function**.
- $\theta = [\theta_0, \theta_1, \ldots, \theta_d]$ is the **weight (parameter) vector** of the model. ($w = [w_0, w_1, \ldots, w_d]$ is also a common notation)
- $\lambda \geq 0$ is a **tuning parameter**.
- $R(\theta)$ is the **regularization term**, **regularizer** or **penalty term**, directly dependent on the values of θ.
- $\lambda\, R(\theta)$ is the **regularization parameter** or **shrinkage penalty**.

There are **two widely used regularizers**: L_1 and L_2.

- $\mathbf{L_2}$ is the **squared norm** of the weight vector θ or, equivalently, the dot product of the θ^T and θ (denoted as $\|\theta\|^2$ or $\|\theta\|_2^2$).

  $$\|\theta\|^2 = \theta^T \cdot \theta = \sum_{j=0}^{d} \theta_j^2 \qquad \text{or} \qquad \|w\|^2 = w^T \cdot w = \sum_{j=0}^{d} w_j^2$$

  This quadratic regularizer is used in **ridge regression** and is know as **weight decay** in ANNs (though the name is usually applied to λ).
- $\mathbf{L_1}$ is the **sum of absolute values** of the parameters (denoted as $\|\theta\|_1$).

  $$\|\theta\|_1 = \sum_{j=0}^{d} |\theta_j| \qquad \text{or} \qquad \|w\|_1 = \sum_{j=0}^{d} |w_j|$$

  L_1 is the regularizer used in **lasso regression**.

The **goal** of adding the regularization term is to stop the weights of the model from reaching large values.

- Minimizing the regularization term implies **minimizing** the values of $\boldsymbol{\theta}$.
- In statististics, regularization methods are a class of **shrinkage methods** because they reduce the value of coefficients of the model.

Regularization limits overfitting by **shrinking the weights** of the model.

- The contributions of **features** to the final prediction are **more similar**.
  - **Lower** model **complexity**.
  - **Less variance**.
- That prevents the **strict memorization** of training data that leads to **overfitting**.

  By penalizing **extreme** weight values, regularization terms prevent **noise** having a big effect in the **parameters** of the model.
- Individual weights θ_j can be even minimized to 0.
  - Regularization can be used as a method of **feature selection** $\Rightarrow$ selected features will have weights bigger than 0.
  - L_1 is more prone than L_2 to **sparsity** (shrinking θ_i values to 0).

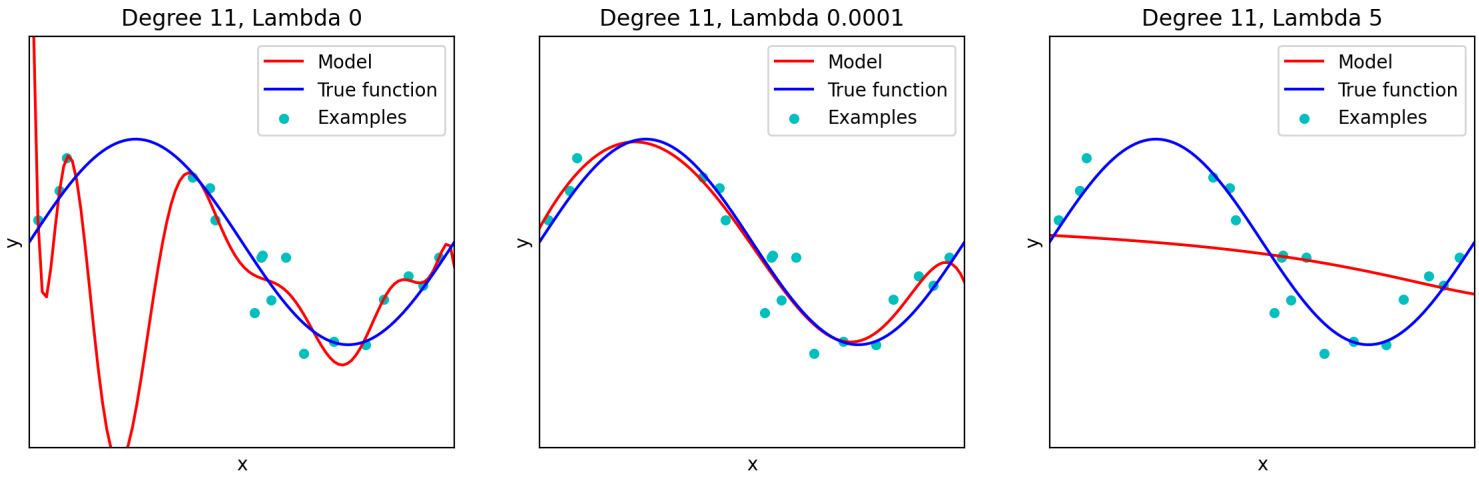
The **tuning parameter** λ governs the relative importance of the regularization term compared to the loss function.

- If $\boldsymbol{\lambda} = \mathbf{0}$ there is **no regularization**.
- The **higher** the value of $\boldsymbol{\lambda}$ the higher the **minimization** of the values of θ.
- If $\boldsymbol{\lambda}$ is **too large**, only very simple models are allowed and we risk **underfitting** the data and introducing **bias**.
- The **value of $\boldsymbol{\lambda}$** is determined during the **training process**, usually by cross validation or using held out data.

E.g., going back to the curve fitting example with $f(x) = \sin(2\pi x)$.

- We try to fit a polynomial of degree 11 using L_2 regularization (ridge regression) and three values of lambda: 0, 10^{-4} and 5.

- We can observe the effects of λ in the complexity and overfitting of the resulting models.
  - $\lambda = 0 \Rightarrow$ no regularization and same overfitting as before.
  - $\lambda = 10^{-4} \Rightarrow$ complexity of the model diminishes resulting in a better fit to the data.
  - $\lambda = 5 \Rightarrow$ model is too simple, underfitting the data.



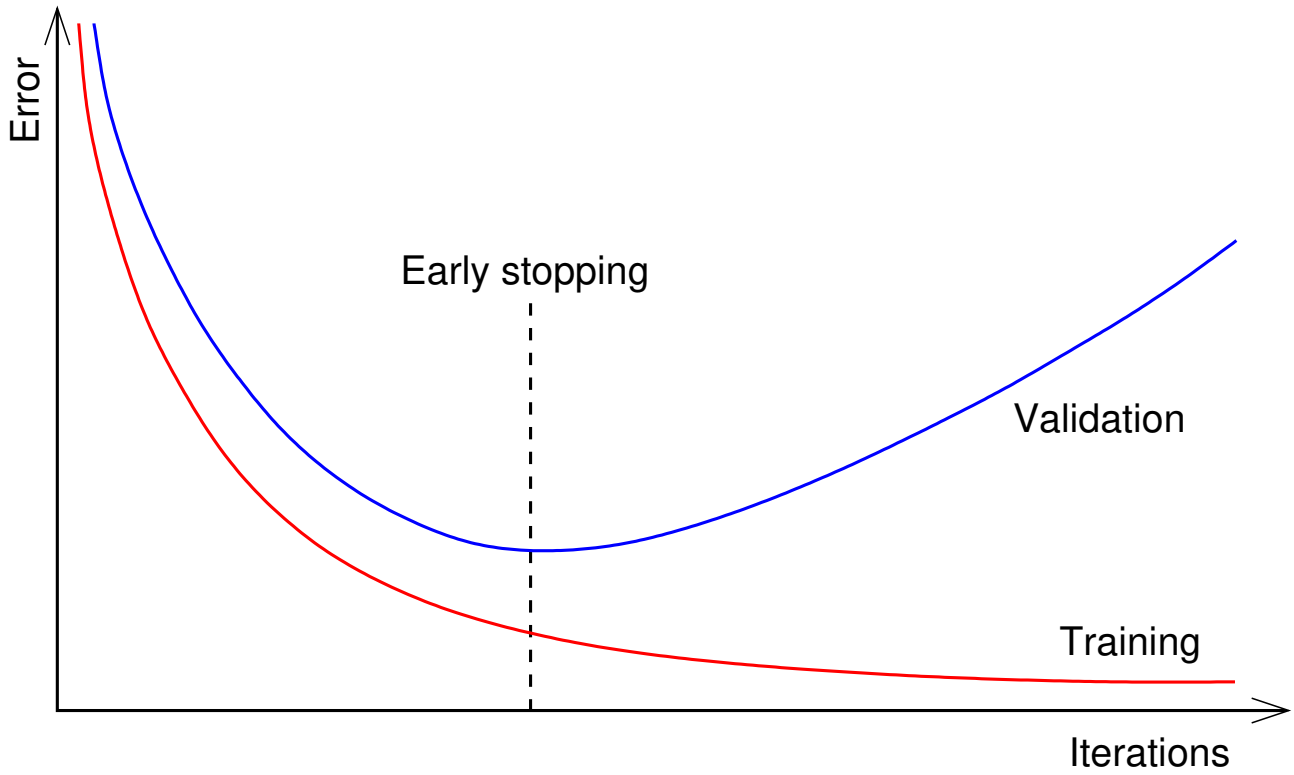
Early stopping

A common **regularization** method for models trained using **iterative** methods.

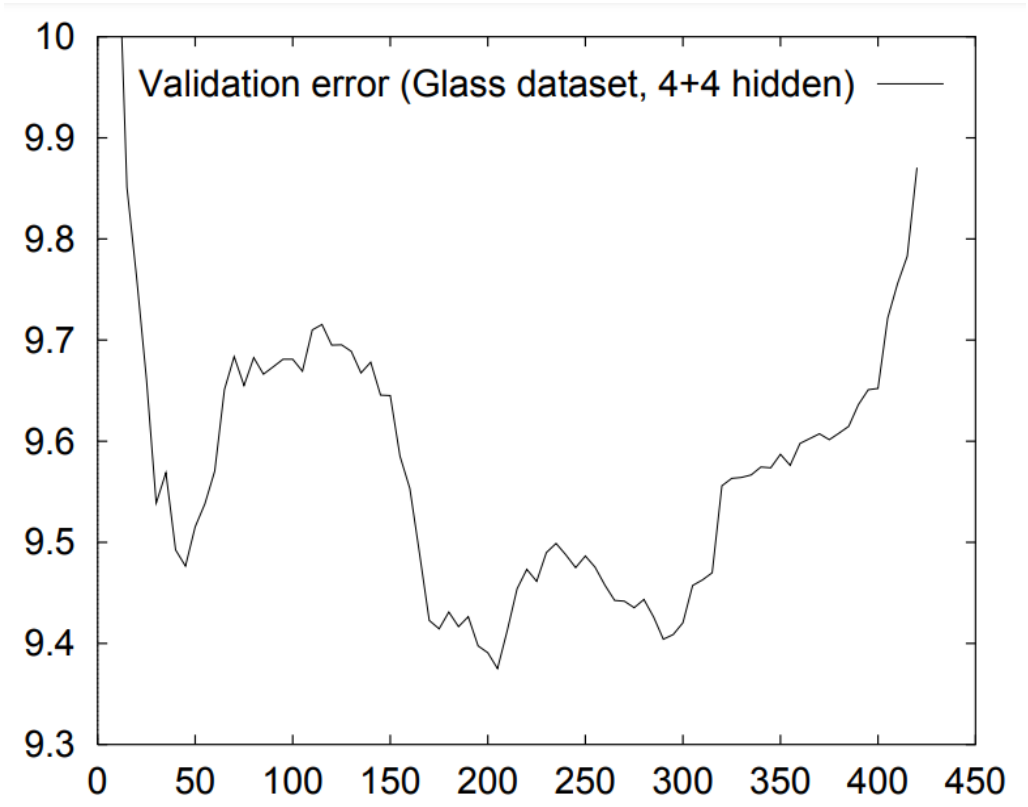
- Very active area of research, especially in **ANNs**.

We keep a complex model, but we **stop training** it before it converges:

- Training is stopped before the model starts **memorizing** the data.
- Early stopping can be used with by any **iterative** learning algorithm (e.g., stochastic gradient descent).
  - We set a **validation dataset** disjoint with respect to the training dataset.
  - The **validation error** of the model is periodically measured during **training** in the validation dataset $\Rightarrow$ usually at the end of each epoch.
  - If the **validation error** starts **growing**, the training stops.
  - The model with the **lowest** validation error is selected.



- **Problem**: this is what the validation error often looks in practice.



Source: L. Prechelt. Early Stopping - but when?

- **Solution**: more sophisticated stopping conditions.

Dropout

A common **regularization** technique used in ANNs.

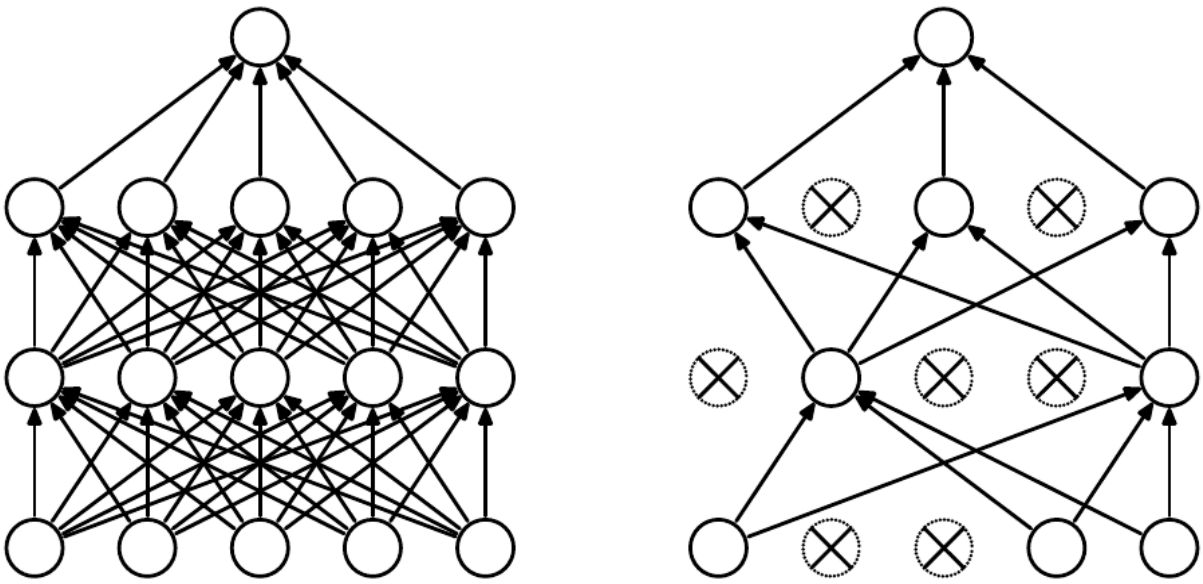
- Can be used along weight decay and early stopping.

Idea: one way to avoid **overfitting** in ANNs is to train several of them and average their outputs through an **ensemble**.

- Ideally involving all possible hyperparameter combinations.
- Problem: computationally **complex** solution.

Dropout "simulates" this behaviour during the **training** of a single ANN, by **randomly dropping** units from the network:

- On a **temporary** basis ⇒ training with the **current batch**.
- Along with their incoming and outgoing **connections** ⇒ weights are not updated.
- A "thinned out" network, with the units that survived dropout, is trained.
- **Stochastic** regularization method ⇒ a sampling of units to drop.
- A lot **less complex** than training an ensemble.



(a) Standard Neural Net (b) After applying dropout.

Source: N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Dropout is usually implemented on a **per layer** basis.

- New **hyperparameter**: the probability of **dropping** out (or retaining) each unit in the layer.
- Layer has **different** nodes and connections at each training step.
- The other units in the layer have to adjust their **weights** to the loss of the dropped ones.
- Outgoing weights are **rescaled** (reduced) using the dropout probability, usually at the end of training.
- Dropout can**not** be used on the **output layer**!

By randomly sampling the layers' output in training, the **capacity** of the network is **reduced**.

- A **wider** network (more nodes per layer) may be **necessary**.

Dropout is **only** used in **training**.

- All units are used when the network is used to make predictions.